

Reporte de Avance: Laboratorio Académico de Seguridad MCP

Proyecto: grupo4-mcp-security

Fase Auditada: Puntos 40 al 44

Equipo: Grupo 4

40. Estudiar arquitectura MCP: clients, servers, transports

La arquitectura del Model Context Protocol (MCP) se basa en un modelo cliente-servidor estandarizado para proveer contexto a modelos de lenguaje (LLMs). Sus componentes principales son:

- **Clients (Clientes):** Aplicaciones o interfaces (como Claude Desktop o herramientas CLI) que interactúan con el usuario y mantienen una conexión con el servidor MCP.
- **Servers (Servidores):** Aplicaciones backend que exponen herramientas (Tools), recursos (Resources) y plantillas (Prompts). Ejecutan las acciones solicitadas de forma local en el host.
- **Transports (Transportes):** El mecanismo de comunicación entre cliente y servidor. En nuestro laboratorio usamos **stdio** (Standard Input/Output), el cual hereda el ciclo de vida del proceso padre.

41. Configurar servidor MCP vulnerable de práctica

Se inicializó un proyecto Node.js/TypeScript desde cero configurado para actuar como un servidor MCP vulnerable con fines educativos.

Repositorio y Comandos Iniciales:

- **URL del repositorio:** `git+https://github.com/ConnieDR8/grupo4-mcp-security.git`
- **Inicialización y Dependencias:**

```
bash
npm init -y
npm install @modelcontextprotocol/server zod
npm install -D typescript @types/node tsx
```

Código e Implementación: El código principal reside en `apps/mcp-lab/src/index.ts`. Se configuró el servidor `grupo4-mcp-lab` (v0.1.0) utilizando el SDK de MCP. Se registraron tres herramientas específicas para el entorno de práctica:

1. `saludar`: Tool segura de referencia que valida sus entradas usando el esquema Zod.
2. `borrar_base_datos_clientes`: Tool destructiva diseñada con una vulnerabilidad crítica de *Authorization Bypass*.
3. `leer_registro_sistema`: Tool de lectura de archivos con una vulnerabilidad crítica de *Path Traversal*.

42. Ejecutar mcp-scanner de Cisco AI Defense

Se utilizó la herramienta `mcp-scanner` desarrollada por Cisco AI Defense para realizar un análisis estático de las definiciones de las herramientas expuestas por el servidor.

Comando ejecutado:

```
bash
mcp-scanner --analyzers yara,prompt_defense --format detailed static --tools
evidence/tools-definition.json
```

Resultados del Escáner:

Preview unavailable

```
text
=== MCP Scanner Detailed Results ===

Scan Target: https://mcp.deepwiki.com/mcp

Tool 1: saludar
Status: completed | Safe: Yes
Analyzer Results (YARA): SAFE

Tool 2: borrar_base_datos_clientes
Status: completed | Safe: Yes
Analyzer Results (YARA): SAFE

Tool 3: leer_registro_sistema
Status: completed | Safe: No
Analyzer Results:
  • yara_analyzer:
    - Severity: HIGH
    - Threat Summary: Detected 1 threat: system manipulation
    - Threat Names: SYSTEM MANIPULATION
```

43. Aplicar checklist de SlowMist punto por punto

Se auditó manualmente el servidor utilizando la estricta metodología del *MCP-Security-Checklist* de SlowMist, documentando el estado de seguridad en `audit/slowmist-report.md`.

Principales hallazgos reportados:

- **[CRÍTICO] Input Validation (1.1):** La herramienta `leer_registro_sistema` sufre de un *Path Traversal* explotable al no sanear ni restringir la entrada enviada por el cliente.
- **[CRÍTICO] Access Control (2.1):** La herramienta `borrar_base_datos_clientes` confía en el campo `rol_usuario` enviado por el cliente en el payload JSON para tomar decisiones de autorización.
- **[MEDIO] Rate Limiting (1.2) y Least Privilege (2.3):** El servidor carece de aislamiento (como Docker o Sandbox), no posee limitación de peticiones (rate limiting) y se ejecuta heredando todos los privilegios de sistema del usuario local.

44. Demostrar vulnerabilidades: authz bypass, injection

Durante esta fase, se explotaron las vulnerabilidades del servidor introduciendo vectores de ataque reales a través de la interfaz del cliente MCP.

A. Authorization Bypass (Control de Acceso Inseguro)

Descripción de la Falla:

La herramienta `borrar_base_datos_clientes` implementa la validación de roles de forma incorrecta: `if (rol_usuario !== 'admin') { ... }`. El servidor solicita al cliente que declare su propio rol en el payload en lugar de verificarlo mediante mecanismos de backend seguros.

Contexto en un entorno real:

Un atacante que usa un cliente malicioso simplemente modificaría el payload JSON para enviar el parámetro `{ "rol_usuario": "admin" }` y así burlar cualquier control de la aplicación, simulando ser un superusuario. En la industria, la identidad siempre debe validarse utilizando un token inyectable seguro (ej. JWT) gestionado por el servidor, nunca confiando en variables estáticas del cliente.

Evidencia de explotación exitosa:

borrar_base_datos_clientes

[VULNERABLE] Elimina todos los registros de clientes. Solo para uso de administradores.

× Read-only

✓ Destructive

× Idempotent

✓ Open-world

rol_usuario *

admin

confirmacion *

☒ Confirmar acción destructiva

Tool-specific Metadata:

Add Pair

No metadata pairs.

Run Tool

Copy Input

Tool Result: Success

```
"¡ÉXITO (Vulnerabilidad)! Base de datos borrada satisfactoriamente. (Authorization Bypass logrado por confiar en el cliente)"
```

Authorization Bypass Exitoso

B. Path Traversal (Lectura arbitraria de archivos)

Descripción de la Falla:

La herramienta `leer_registro_sistema` invoca `fs.readFileSync(ruta_archivo)` usando el parámetro exacto provisto por el usuario, sin validar que la ruta corresponda exclusivamente al directorio de logs permitido, permitiendo secuencias de escape de directorios relativas.

Contexto en un entorno real:

Un atacante introduciría rutas como `../../../../etc/passwd` (en Linux) o `../../../../package.json` para extraer archivos sensibles del sistema operativo huésped. Esta vulnerabilidad permitiría la filtración masiva de credenciales, claves API o el código fuente del proyecto de la empresa que hospeda el servidor MCP.

Evidencia de explotación exitosa (Lectura de package.json):

leer_registro_sistema

[VULNERABLE] Lee un archivo de registro del sistema. Se asume que el usuario enviará un log relativo válido.

✖ Read-only

✔ Destructive

✖ Idempotent

✔ Open-world

ruta_archivo *

package.json

Tool-specific Metadata:

No metadata pairs.

Add Pair

Run Tool

Copy Input

Tool Result: Success

"Contenido leído exitosamente (Vulnerabilidad Path Traversal lograda):"

```
{
  "name": "grupo4-mcp-security",
  "version": "0.1.0",
  "description": "Laboratorio académico de seguridad para servidores MCP",
  "scripts": {
    "dev": "tsx apps/mcp-lab/src/index.ts",
    "typecheck": "tsc --noEmit"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/ConnieDR8/grupo4-mcp-security.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
```

Path Traversal Exitoso

Análisis de Discrepancias: Escáner Automatizado vs Auditoría Manual

Una conclusión central de este laboratorio es observar la discrepancia técnica entre las herramientas automatizadas y las revisiones manuales:

- **Por qué `mcp-scanner` detectó el Path Traversal:**

El escáner de Cisco utiliza un motor de reglas (YARA) que busca firmas estáticas conocidas. Al

detectar el uso de funciones del File System combinadas con parámetros de entrada (`ruta`), identifica correctamente un patrón de "Manipulación del Sistema" marcándolo como **ALTO**.

- **Por qué `mcp-scanner` falló en detectar el Authorization Bypass:**

A pesar de ser una falla crítica, el escáner marcó a `borrar_base_datos_clientes` como **Seguro**. Esto se debe a que un *Authorization Bypass* es una falla de **Lógica de Negocio**. El escáner estático ve una validación condicional normal de un string en el código y carece de contexto semántico para comprender que "confiar en el string del usuario para otorgar acceso" constituye un riesgo.

Conclusión:

Esto corrobora por qué auditorías automatizadas nunca deben ser el único filtro de seguridad. Las revisiones manuales exhaustivas con metodologías estructuradas como SlowMist son vitales para identificar riesgos conceptuales y lógicos que las firmas estáticas no pueden comprender.